# **Final Project**:
# Nexus (i.e., Reddit-like platform)

- **Name**: Harsh Patel
- **Hawk ID (A-Number #)**: A20369913
- **Subject**: ITMD 555 Intelligent Device Apps Android
- **Professor**: James Papademas
- **Test Emulator**: Pixel 7 pro API 35
- **Language**: Java

# Understanding the Idea Behind the Project

**APIs**:
- **Google Firebase APIs**
  - Firebase Authentication
  - Cloud Firestore
- **Web-based routing API** for cross-platform sharing (i.e., currently on localhost)

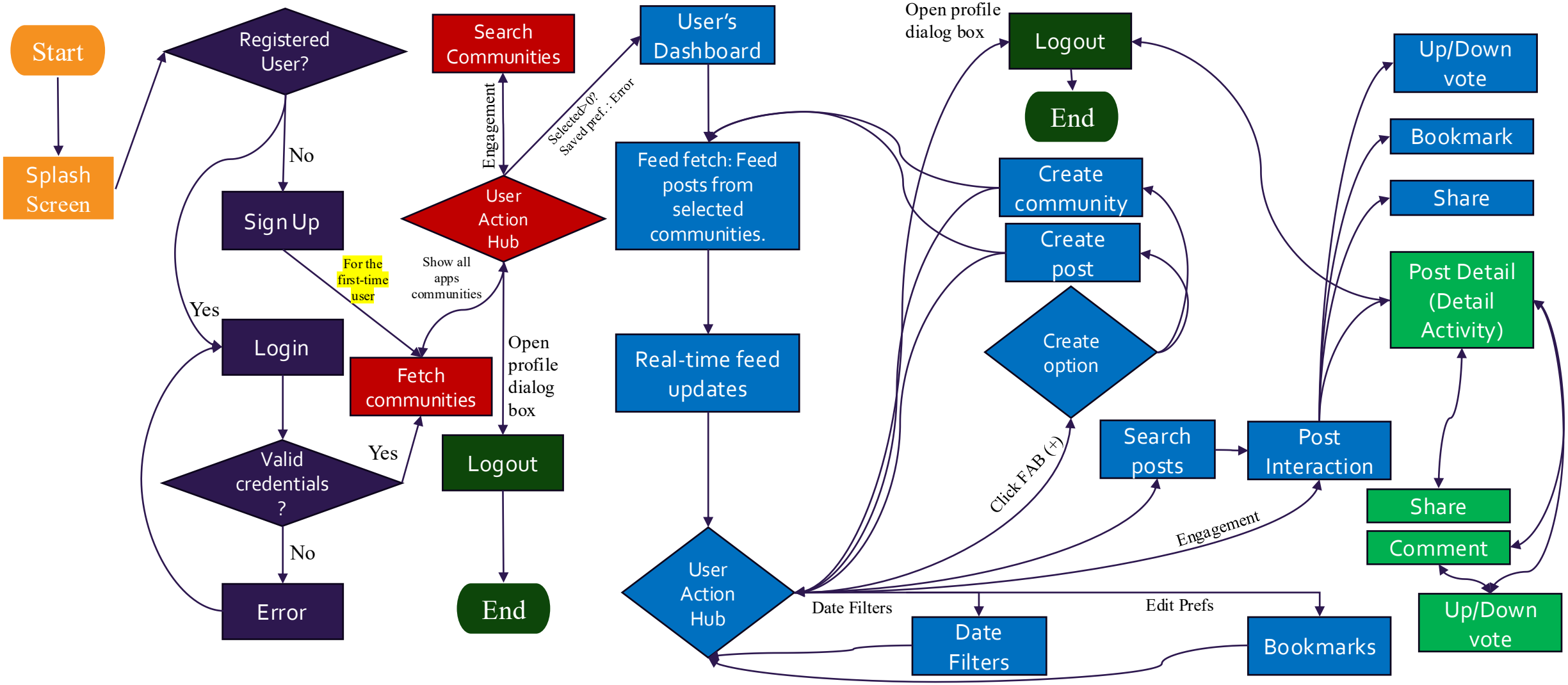
- Nexus is a **community platform, like Reddit**, that bridges fragmented social discussions to deliver high-quality information and meaningful discourse.
- Users can **join or start community pages, create posts on any community, and view posts from others**.
  - i.e., Business, Entertainment, Health, Science, Sports, Technology, Gaming, Traveling, MacBook, or any random topic, etc.

- **Groups**: Users begin by customizing their post feed through filtration (e.g., select/deselect Creative Design, Retro). This ensures the "Nexus Dashboard" is relevant from the very first login.

- **Current Features**:
  - Like, dislike (posts/comments), comments (based on threads), and share posts.
  - Bookmark posts with “Recaps” for quick access later.
  - Search content directly from the main feed.
  - Community-based feed with filtering options.
  - Share posts with other users easily.
  - Create and engage across multiple communities.
  - Backend powered by Firebase (NoSQL database).

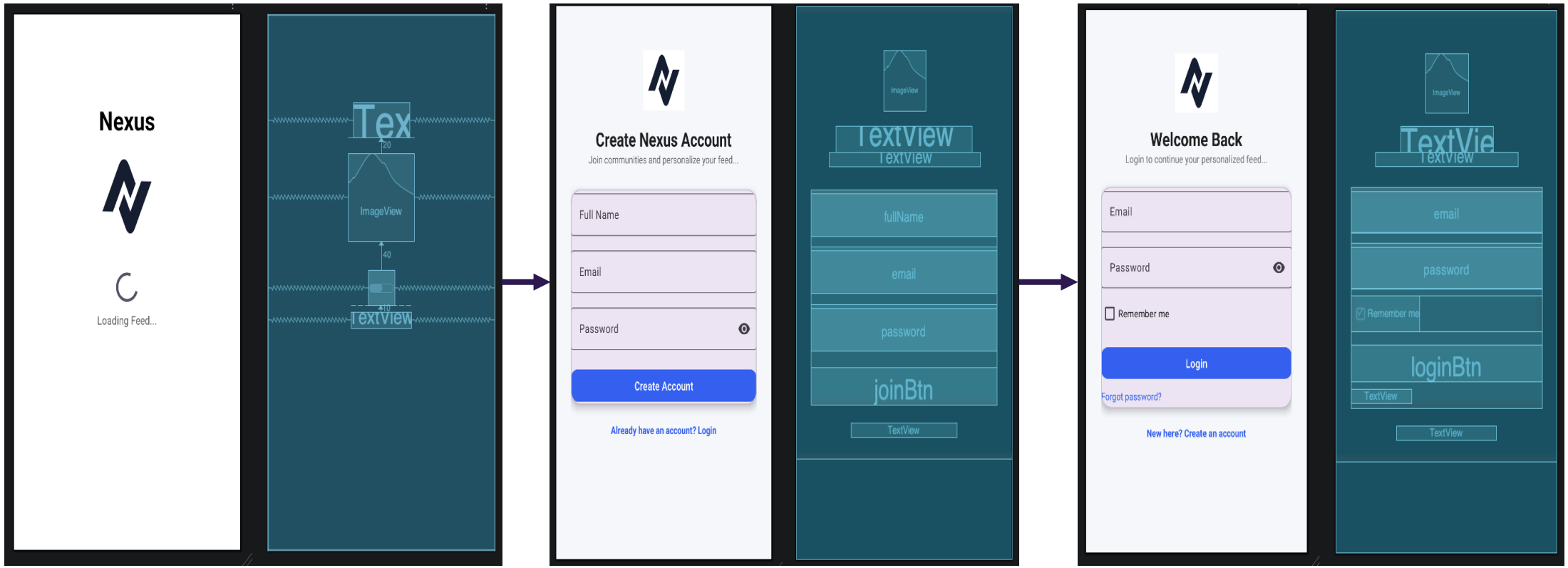
- **Firebase**:
  - Firebase Authentication (for user login & identity management)
  - Cloud Firestore (for database operations like storing users, likes, comments, etc.)

# Implementation flowchart - 1



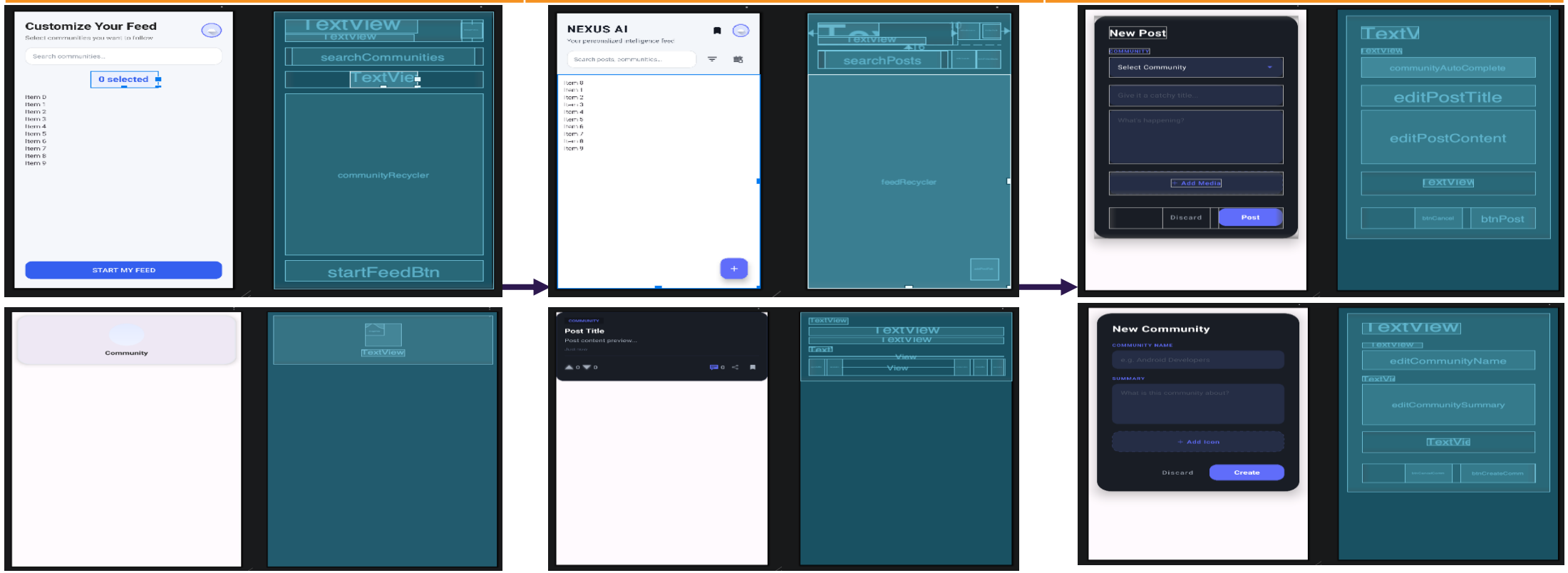
# Implementation flowchart-02 (U/I part-01)

| Splash Screen | Sign up | Sign in/Login |
|---|---|---|



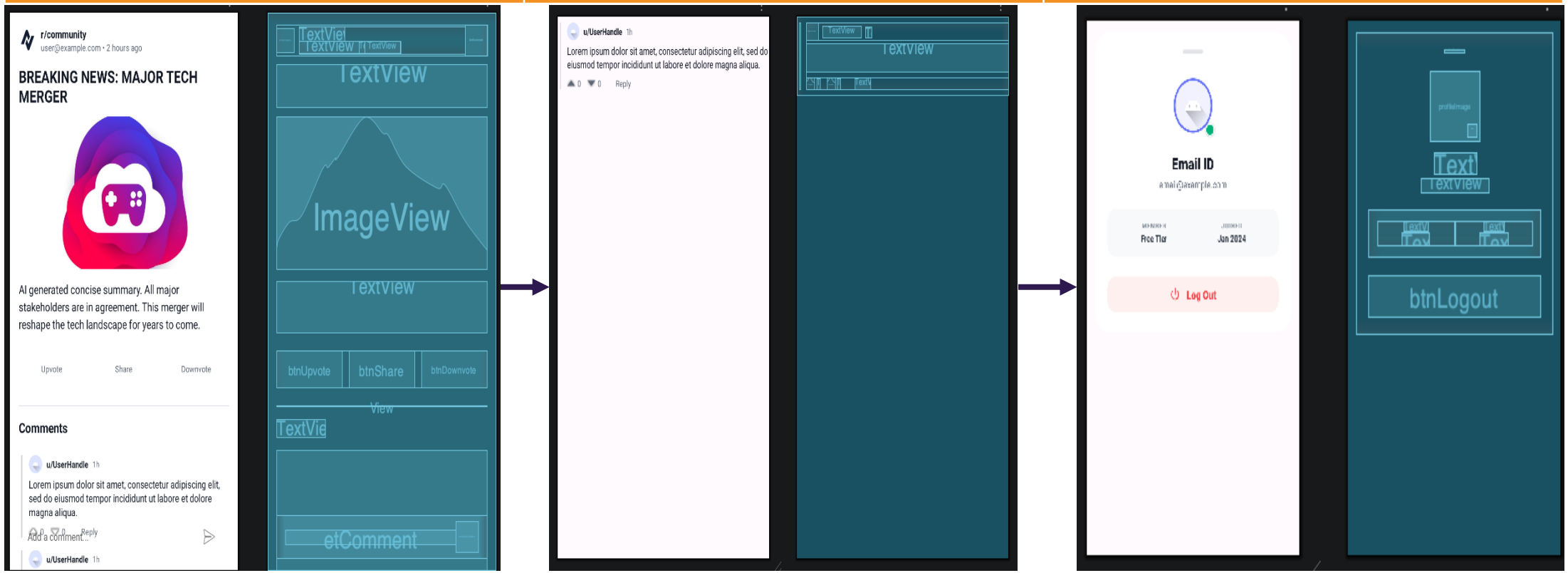
# Implementation flowchart-02 (U/I part-02)

| Topic selection activity | Activity Dashboard - 01 | Activity Dashboard - 02 |
|---|---|---|



# Implementation flowchart-02 (U/I part-03)

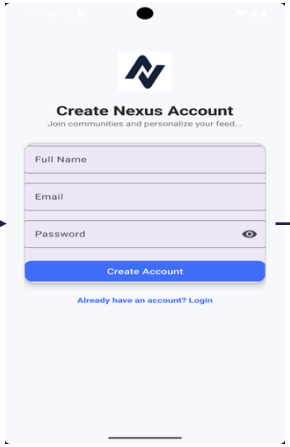
| Detailed Post Activity - 01 | Detailed Post Activity - 02 | Logout (i.e., from any Page) |
| --- | --- | --- |



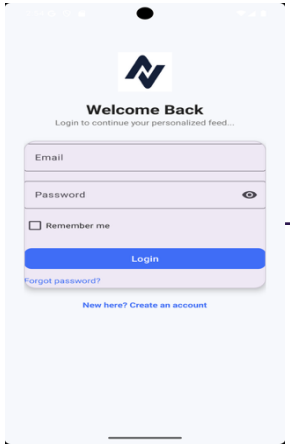
# Finalized App Screenshots

Splash Screen

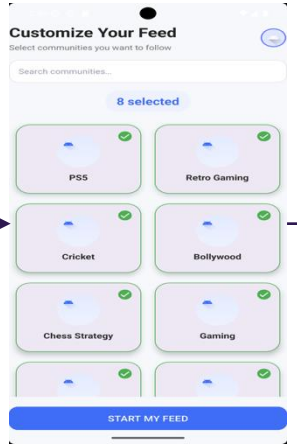
Sign up



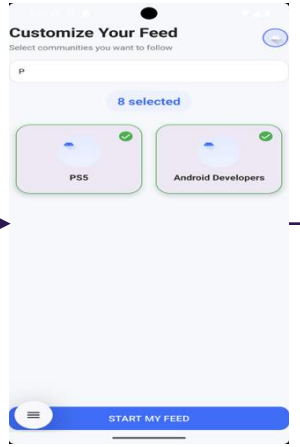

Log in



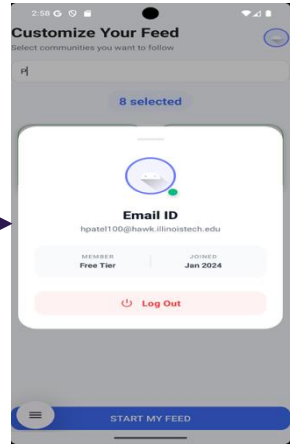

Select community



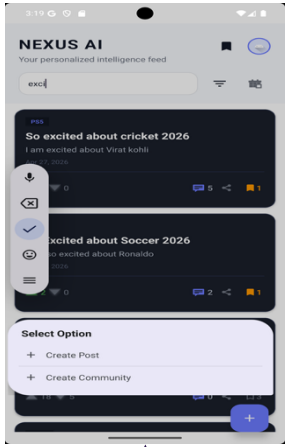

Find community



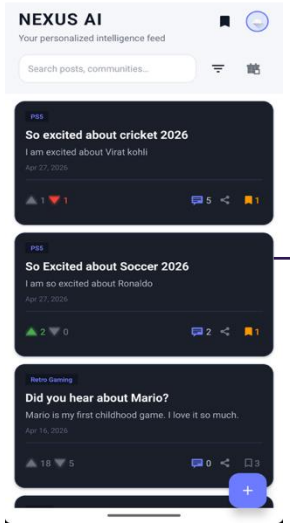

User's profile



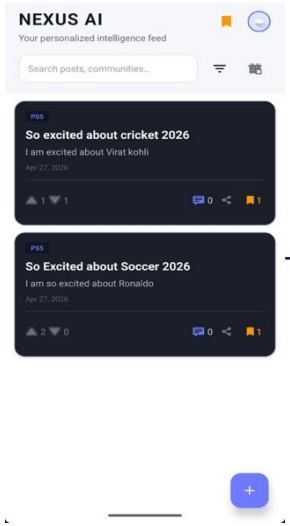

Create a new post OR community




Main Dashboard

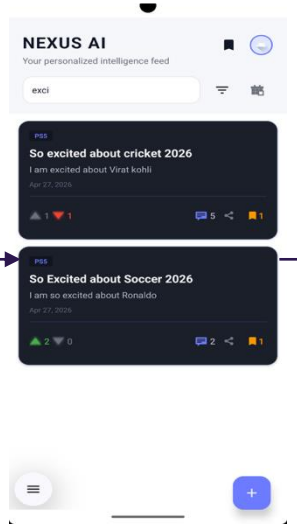



Bookmarked posts

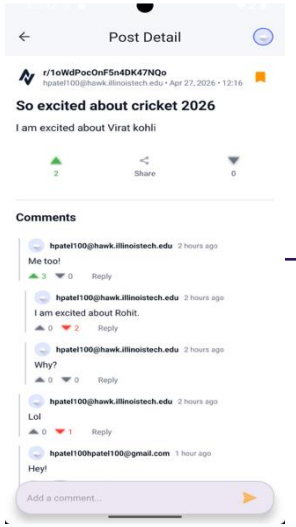



Filter by date

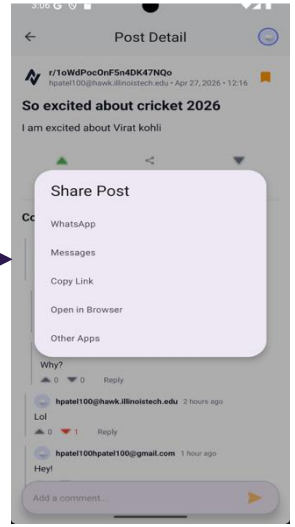
Search posts



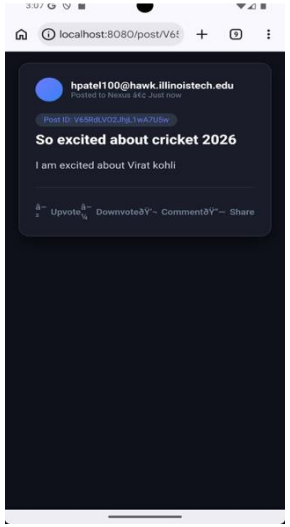

Detailed Post




Share the post

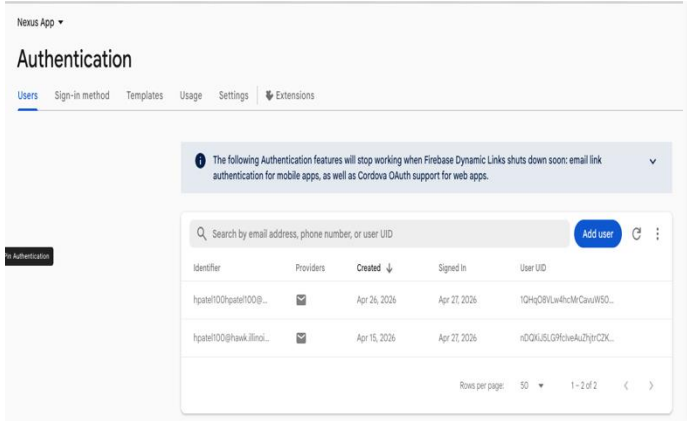



Shared post

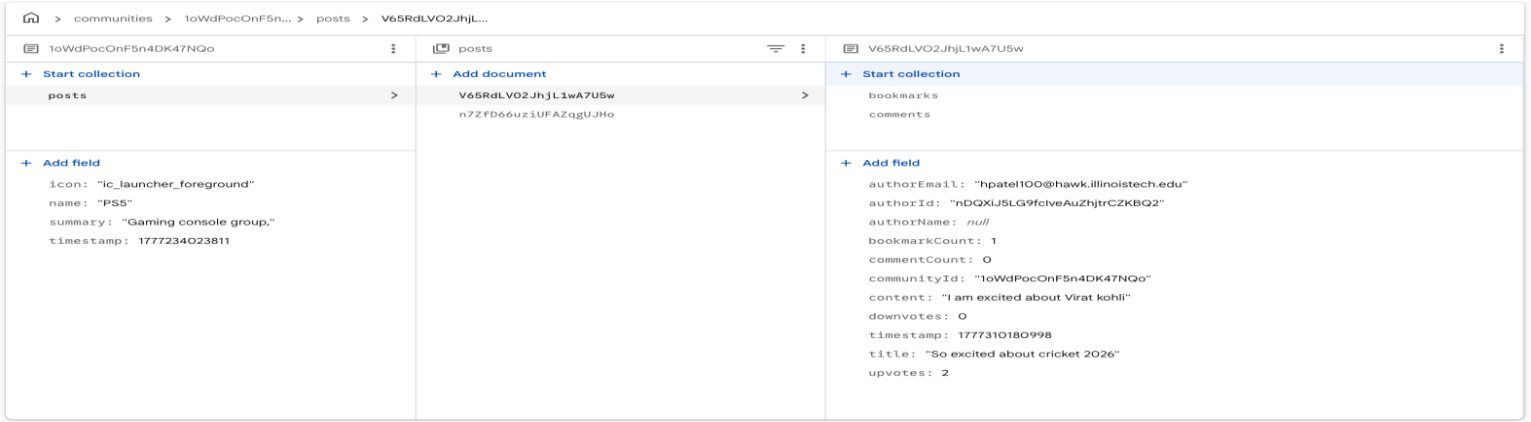


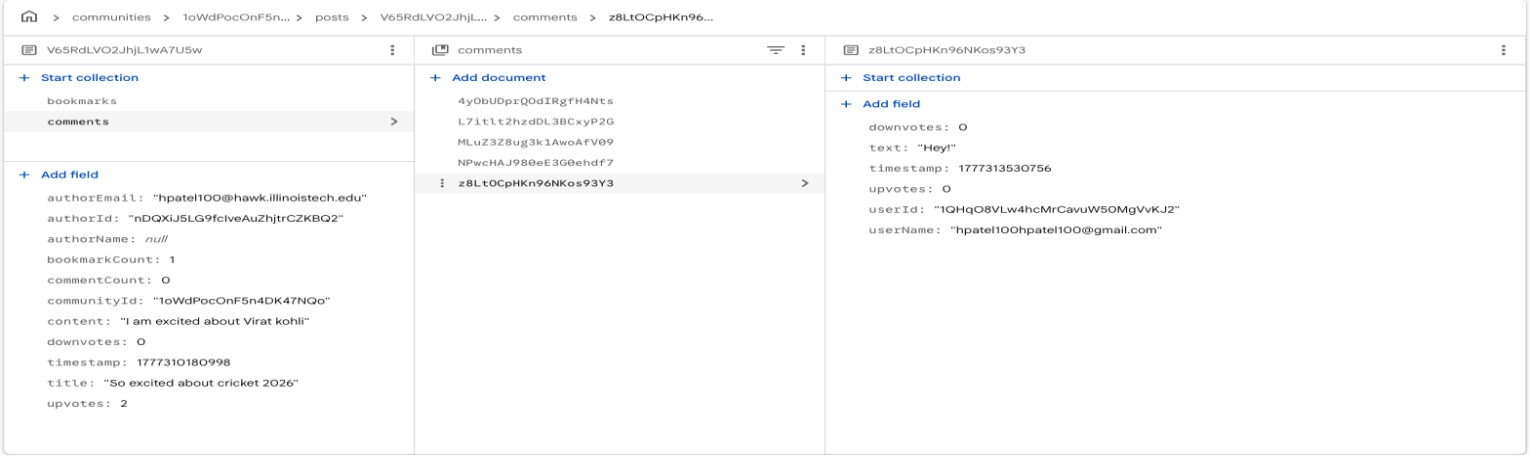
# Firebase DB Implementation

## User's Authentication




## Communities → Posts → Comments + Bookmarks



# Problem areas encountered:

- **Like/Dislike System:** Stores individual user reactions in a database for AI-based analysis and summarization.
- **Comment System:** Saves threaded comments and tracks likes on each comment.
- **Local Deployment:** Hosted on localhost for development and testing (no external server required currently).
- **U/I Design:** Focus on building and refining user-friendly interface designs.
- **Emulator Selection:** Choosing a lightweight emulator optimized for smooth performance on a MacBook (2020 model).

## Future App modifications and tweak suggestions:

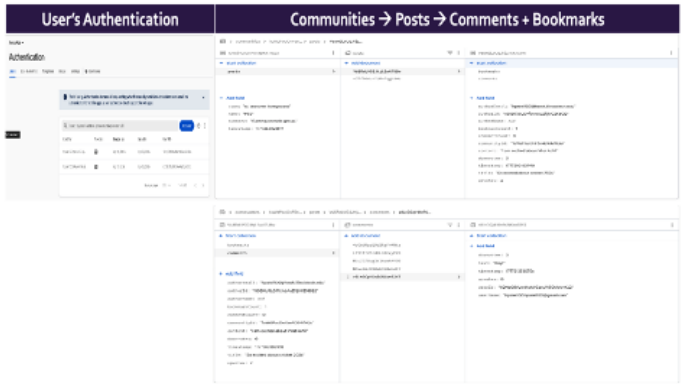
- **Live Deployment**: Hosting the app and collecting real user feedback and reviews.
- **Performance Optimization**: Integrating native libraries for faster and more efficient access.
- **U/I Revamp**: Designing a fresh, modern interface inspired by Reddit-style user experience.

# APIs used proofs and Extra Credits

**API-1 (user login/signup) + API-2 (storing and retrieving data)**

Firebase DB implementation

Code of google-services.json is in the peoject code file.




**API-1**: You can see User authentic in Firebase.

**API-2**: You can also see collecti subcollections of each collections:

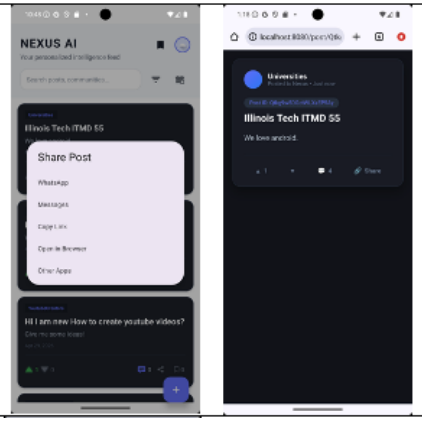
- Communities
  - Posts
    - Book
    - Com

**Proof**:
DashboardActivity.class + PostDetailActivity.class's FirebaseFirestore.getInstance() method.

**API-3 (Custom Python Web API)**

local server.py backend

(i.e., people share a post on reddit and can see it "on broswer" without interacting it, so I created my own rest api to see it on browser so user can see it without logged it).




**The Custom Backend API (server.py)**
A Python script that starts a s http://localhost:8080.
Endpoint: it listens for requests at /post/{post

Logic: It accepts query parameters (like titl and author) and dynamically generates a f "Web View" of a post.

Language: Python (using http.ser PostDetailActivity.java (and FeedAdapter.ja

**It's a REST API (A custom web API I cre**
1. Endpoint: /post/
2. Protocol: HTTP (running on port 8080).
3. Communication: Your Android app "tal server to generate shareable links.

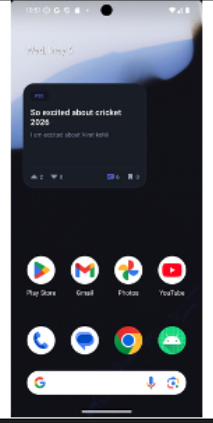
Even you can copy localhost link and Macbook broswer.

**API-4 + (Extra credit-1)**

**Background/Widget Features showing most interacted post**

**Feature-1:**
*(Widget features, Users can see the most interacted post from activity dashboard in 'Nexus' app widget)*

*Btw, You need to add Nexus widget in device.*



**To see it before you share, you need to**
1. Start server program, and

**On 1st terminal screen:**
*python3 server.py*

2. Allocate 8080 port to emulator

**On 2nd terminal screen:**
*adb reverse tcp:8080 tcp:8080*

Every time the widget updates, it sets AlarmManager timer to trigger itself again 60,000ms (1 minute). This creates a continuous where the widget pulls fresh data from Fires every minute.

When a push notification arrives with a payload, it calls updateWidget().This triggers WidgetUpdateWorker, which forces the widge refresh immediately without waiting for the minute timer.

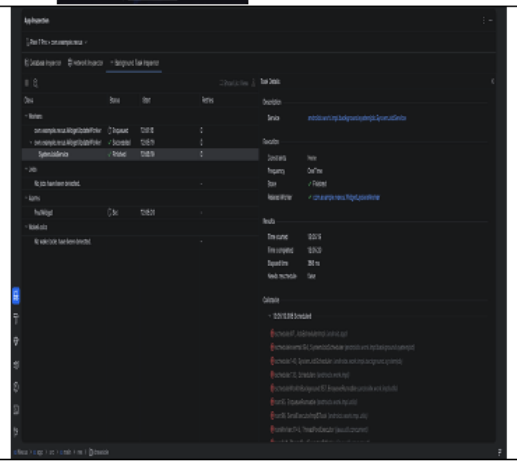
**NOTE**: However, in some Android devices li this to a minimum of 30 minutes to save battery

**Note**:
1. You must be logged in.
2. You have selected communities.
3. And if it's a single post for sela communities.

**(Extra credit-2)**

**Service-1+2: Background Task Management**

**API Utilized: Android Jetpack WorkManager API (androidx.work) + AlarmManager**



**Real-time Monitoring**: Using the Android St Background Task Inspector, execution was veri on a live device.

The inspector confirms that WidgetUpdateWorker successfully transitio from an "Enqueued" state to "Succeeded."

The background task was efficiently handled by system's JobScheduler, completing the update c in 386ms.

The screenshot provides a full audit trail, show the exact time of execution (12:05:19) and underlying system callstacks that managed the t

**Proof**:
WidgetUpdateWorker.java class and triggerWidgetUpdate() method DashboardActivity

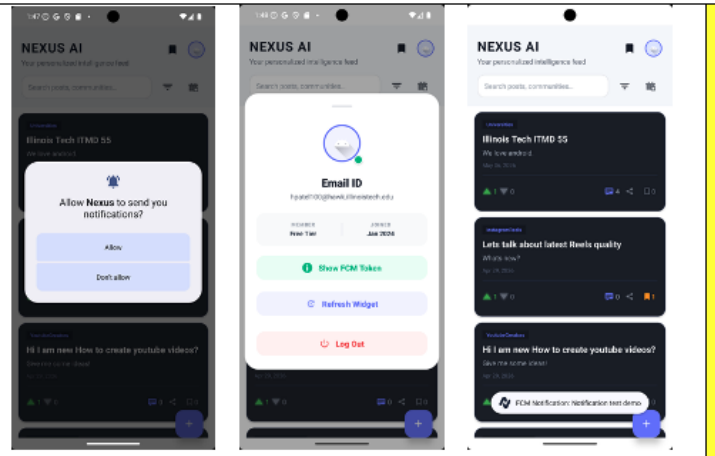
# API used + Extra credits
# Updated on 6th May (After presentation)

# API used + Extra credits Updated on 6th May (After presentation)

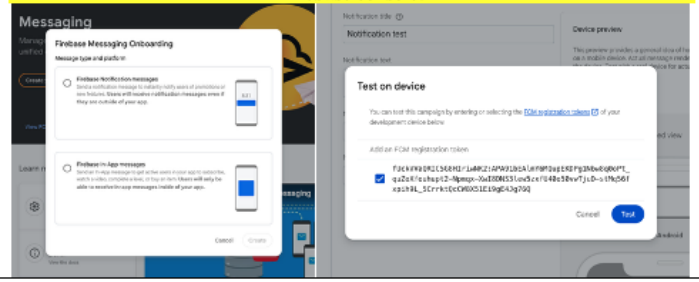
**(Extra credit-3)**

**Service-3: Firebase Cloud Messaging**

**(Background services)**




**Firebase console:**




**MyFirebaseMessagingService**: Created a custom service to handle incoming messages.

**Foreground Handling**: Implemented onMessageReceived to display a Toast notification when the app is open

FirebaseMessaging.getInstance().getToken() to fetch the unique device registration token.

Added a one-tap "Copy" feature to the token dialog, making it easy to grab the token for testing via the Firebase Console or curl.

Verified that messages sent from the Firebase Console (V1 API) are successfully received by the device.

Visual confirmation as well on movile devie.

**i.e., Copy token from show FCm Token from user's profile.**

**(Extra credit-4)**

**Github commits**

GitHub Link: https://github.com/harshjpatel/ITMD555Project_Nexus | Professor (GitHub username: @igitiit) got added as a collaborator already.